Integrate the Model

2026-05-06 · cheerful mango Haubentaucher

Train hard, ship easy.

The Why

Block 3 left us with a reproducible environment on both machines. The virtual environment is ready, the dependencies are pinned, the project foundation is in place. A foundation that would work for all sorts of different projects by the way. Let's get our project some specific functionality.

We build PixelWise. This block is the first that produces application logic in the form of a neural network, in the following referred to as model. We assume the model is already trained and available, and our task is to integrate it into the codebase and make it callable.

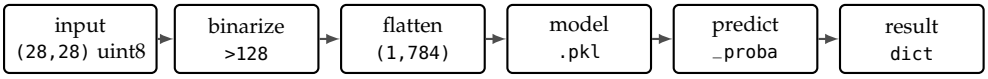

Figure 1:
The inference pipeline for one image: a raw `(28,28)` grayscale array is binarised at threshold 128, flattened into a `(1,784)` row vector, fed to the loaded `.pkl` model, and turned into a result dictionary by `predict_proba`.

The gap between a trained model and a working system is larger than most students expect. A `.pkl` file sitting in a folder is not a product. It needs to be loaded safely, called with the exact input format it was trained on, and wrapped in an interface the rest of the application can depend on, suffice to say validated. The perfect mdoel however, is not useful if it is not integrated into the system, so it can be exposed and put to work.

This block bridges that gap. We take a pre-trained digit classifier, integrate it into the PixelWise codebase, and verify end to end that it is functional before any API or frontend code exists. The pipeline you build here, model file in, prediction dictionary out, is the contract every later block consumes without modification.

Hands On Experience

Consider this scenario: a colleague hands you a trained model file and a one-line note, "ship it". This block teaches you how to, and why you should, ask the right questions to your colleague.

This fourth block integrates a trained model into PixelWise and builds the inference layer. By the end you will have

- understood what a model file is and why it is not source code,
- loaded the model safely, learned about and verified its class contract at startup,
- implemented a batch-first inference interface in `app/classifier.py`,
- written a smoke test that proves the integration honours the contract,
- and documented the model's capabilities and limitations in a model card.

Verification and validation are two different questions. Verification asks whether the system was built right: does the integration return what the model is trained to produce, with the expected shape, dtype, and class set? Validation asks whether the right system was built: is this model the right choice for the problem in the first place and is the performance sufficient? This block is squarely about verification. Validation belongs to pre-integration.

What a Model File Actually Is

A `.pkl` FILE IS A BUILD ARTEFACT, not source code. You distribute it; you do not commit it to your repo. Just as a compiled binary is produced by a compiler, a model file is produced by a training script. The training script and its configuration is the source of truth: It takes data, runs an algorithm, and writes weights, hyperparameters, and any preprocessing into a single serialised object.

VERSIONING MAKES THAT CONTRACT DURABLE. Files carry a version in the filename or in metadata shipped alongside. A retrain on more data bumps the patch version; an architecture change or a different class set bumps the major version. Integration code pins to a specific version and updates deliberately, the same discipline you would apply to any external API. Open source tooling that automates this includes DVC for git-style data and model tracking, MLflow and ClearML as self-hostable experiment registries, Aim as a lighter-weight tracker, and the Hugging Face Hub itself, which is free for public repositories and versions every push as a git revision.

HUGGING FACE IS THE GITHUB OF MODELS. The Hugging Face Hub at `https://huggingface.co` hosts hundreds of thousands of public model repositories, each with versioned weights, a model card describing it. Downloading a specific revision is a one-line call against the `huggingface_hub` library. The same workflow runs against private repositories with a token, which is how many companies share proprietary models internally. For this course the artefact lives in the course-maintained model repository at `https://github.com/schutera/pixelwise-model`, with a tagged release per model version and the model card sitting next to it.

LOADING A `.pkl` FROM AN UNTRUSTED SOURCE CAN EXECUTE ARBITRARY CODE. `pickle` and `joblib` deserialise objects by reconstructing them, including any code embedded in the file. This is convenient when you produced the file yourself; it is dangerous when the file came from somewhere you do not control. Treat `.pkl` files the way you treat executables: only run what you trust.

TRAINING FROM SCRATCH EVERY TIME IS UNREALISTIC. Even our small classifier takes seconds; a production scale model takes days of GPU time and terabytes of data. The training team tunes, the inference team integrates, and the artefact is the contract between them.

PULLING A MODEL FROM THE HUB.

```
from huggingface_hub import
hf_hub_download
path = hf_hub_download(
  repo_id="distilbert-base-uncased",
  filename="model.safetensors",
  revision="v1.0",
)
```

THE TRAINING SCRIPT `train.py` lives in the course `pixelwise-model` repo alongside the artefact it produces, not in PixelWise. The artefact and the script that produced it travel together, so any consumer of a tagged release can audit how that exact `.pkl` came to be. Students who want to understand how the artefact was produced can run it: MNIST downloads automatically via `sklearn.datasets.fetch_openml`, and training takes ∼30 seconds on CPU. Further reading on training practices: Karpathy, A Recipe for Training Neural Networks: `https://karpathy.github.io/2019/04/25/recipe/` Google, Rules of Machine Learning: `https://developers.google.com/machine-learning/guides/rules-of-ml`

Exercise: Pull the Model into Dev

TREAT THE MODEL ARTEFACT AS SOMETHING YOU LOAD, not something you commit to PixelWise. The course hosts `digit_classifier_v1.pkl` as a tagged release in the central `pixelwise-model` repo; you pull it into `models/` of the main project, you do not redistribute it.

PIN THE MODEL VERSION in PixelWise. Add to `.env.example` so others know which release the integration expects:

```
MODEL_REPO=schutera/pixelwise-model
MODEL_VERSION=v1.0
```

Then download the tagged artefact into `models/` using the GitHub CLI; the same call works locally and inside CI:

```
mkdir -p models/
gh release download "$MODEL_VERSION" \
    --repo "$MODEL_REPO" \
    --dir models/
ls models/
git status
```

CONFIRM THE ARTEFACT LOADS in a Python shell on dev:

```
source .venv/bin/activate
python3
>>> import joblib
>>> model = joblib.load("models/digit_classifier_v1.pkl")
>>> type(model)
```

If `joblib.load` returns a `Pipeline` object without raising, the artefact is sitting at `models/digit_classifier_v1.pkl` on your dev VM, ready to be scrutinised next.

CATCHING UP VIA THE REFERENCE TAG. The course maintains a public reference repository at https://github.com/schutera/pixelwise with a tag at the end of every block. `v0.2` marks the end of Block 3; `v0.3` will mark the end of this block. If you joined late or your repository has drifted, reset to the previous tag before starting:

```
git fetch origin
git reset --hard v0.2
```

This rewrites your working tree to match the end of Block 3 exactly, including `setup-server.sh`, `requirements.txt`, and `.env.example`.

THE COURSE ARTEFACT. at https://github.com/schutera/pixelwise-model `digit_classifier_v1.pkl` is a LogisticRegression pipeline trained on MNIST digits 1 to 9 (9 classes, ∼54,000 training samples, public domain). Class 0 is intentionally held back; students can add it later as a v2 release without collecting any new data.

WHY PULL FROM A SEPARATE REPO? The model has its own release cadence and its own contract. Pinning a specific tag in PixelWise's `.env` makes the integration explicit: a v2 only takes effect when the integration code chooses to. The optional exercise at the end of the block walks through publishing your own v2 to a fork of the model repo.

IF `digit_classifier_v1.pkl` APPEARS IN `git status`, your `.gitignore` from Block 3 needs an update. Add `models/*.pkl` on its own line and rerun `git status`. A model file accidentally committed once is in the history forever, even if the next commit deletes it.

The Model Contract and Its Card

Every model was trained on inputs in a specific format. The format is part of the contract: a 28-by-28 greyscale image, pixel values in $[0, 1]$, flattened to 784 features. Most violations of this contract are loud: a wrong shape raises `ValueError` on the first call, a wrong dtype trips an assertion in the pipeline, a missing class never appears in the predictions at all. The integration layer's job is to catch these before the model reaches production.

Silent failures live in the gap between syntactic and semantic correctness. Same dtype, same dimensions, but pixel range $[0, 255]$ instead of $[0, 1]$, or raw greyscale instead of binarised. The model returns a class label and a confidence score with no way to tell that the input did not come from the training distribution. This is the train/serve skew bug class, and the only fix is an explicit preprocessing step that mirrors training.

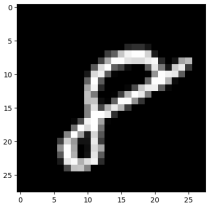

Figure 2: An MNIST sample: a handwritten digit on a 28×28 greyscale grid, the input shape every consumer of `digit_classifier_v1` must respect.

The MNIST model expects:

- a 28×28 greyscale image,
- pixel values normalised to $[0, 1]$,
- flattened to a 784-element vector.

The sklearn Pipeline bundles preprocessing and model into one serialised object that is itself the contents of `digit_classifier_v1.pkl`. `joblib.load` returns the whole Pipeline, preprocessing steps included, so loading the file is loading the preprocessing, so we reduce steps a maintainer might forget; adhering the contract the training script established.

Documentation: scikit-learn, joblib, NumPy.

The model card is where you write the contract down. A model card answers four questions any consumer of the model should know the answer to before they trust it: where did the data come from, what can the model do, what does it fail at, and what is it intended for. The contract that runtime code enforces and the card that humans read are two views of the same thing.

Model cards as a concept were proposed by Mitchell et al. in 2018 and are now standard at Google, HuggingFace, and increasingly across the industry. Model Cards (Google), HuggingFace Model Cards.

For PixelWise the v1 model card lives in the central `pixelwise-model` repository alongside the `.pkl` it documents. The card travels with each tagged release, so anyone who pulls `v1.0` also gets the card describing what `v1.0` can and cannot do:

```
# MODELCARD.md: digit_classifier_v1

## Training Data
MNIST digits 1-9, ~54k train / ~9k test, public domain.
Class 0 withheld intentionally.

## Capabilities
Predict handwritten digits 1-9.
Expected accuracy: ~92% (LogisticRegression baseline).

## Known Failures
6/9 confusion, 3/8 confusion, messy or rotated digits.

## Intended Use
28x28 canvas drawings.
Out of scope: photos, non-digit characters.
```

THE CARD DOES NOT REPLACE THE RUNTIME CHECKS built into the loader. Documentation tells humans what to expect; the assertion at module load tells the program what to expect. Both are needed. Documentation drifts; assertions cannot.

THE CARD IS A CONTRACT, like any other. When v2 is published, the card is updated alongside it; the version number, the training data section, and the capabilities all change in lockstep. Traceability across versions is the entire reason the card is a separate file.

Exercise: Scrutinize the Pipeline and Write the Card

VERIFY WHAT THE PIPELINE DOES rather than rebuild it, then record what you found in a model card of your own. The preprocessing for the model is part of the artefact, not something you re-derive on the caller side; your job here is to confirm what is inside the `.pkl` matches what the canonical model card promises, and to write that contract down for your project.

GET AN MNIST SAMPLE ONTO YOUR DEV MACHINE. The dataset is available through scikit-learn and downloads on first access:

```
from sklearn.datasets import fetch_openml
X, y = fetch_openml(
    "mnist_784", version=1,
    return_X_y=True, as_frame=False,
)
sample = X[0].reshape(28, 28).astype("uint8")
label = y[0]
```

You now have one ground-truth example to scrutinise the pipeline against.

OPEN THE PIPELINE AND LOOK AT ITS PARTS. The Pipeline object exposes its named steps as a dictionary, and the classifier exposes the classes it knows:

```
import joblib
model = joblib.load("models/digit_classifier_v1.pkl")
for name, step in model.named_steps.items():
    print(name, type(step).__name__, step)
print("classes:", model.classes_)
print("n_features expected:", model.n_features_in_)
```

Answer the following from the printout alone:

- Which preprocessing steps are inside the pipeline, and which are the caller's responsibility?
- Does the class set match what the model card says?
- What input shape and dtype does the classifier expect?

A mismatch between what the pipeline contains and what the documentation promises is exactly the kind of contract violation the integration layer exists to catch. The next exercise turns this manual check into a smoke test that runs on every commit.

THE POINT IS VERIFICATION. You are not reproducing the model's preprocessing; you are confirming the artefact behaves the way the model card claims. A pipeline that contains an unexpected step, or is missing one you expected, is a contract violation you want to catch here, not later.

NOW WRITE THE CONTRACT DOWN. Create `models/MODELCARD.md` in PixelWise with the four sections from the theory section: training data, capabilities, known failures, intended use. Use the values *you* measured in the steps above, not the round figures from the canonical card.

The card lives in PixelWise even though the `.pkl` does not. Documentation that travels with the integration code is the part of the contract that survives a model-file deletion or a registry outage.

COMPARE AGAINST THE CANONICAL CARD at schutera/pixelwise-model. Yours is a working copy that documents your own measurements; the canonical card is what consumers of the release see. A v2 you publish later (in the optional exercise) will carry your card as its release-time card.

Designing the Inference Interface

BEFORE WRITING ANY CODE, design the contract. Start with the class roster, then the function signatures, then the loading strategy. Each is a decision that propagates: every later block reads from this interface, and every change to it ripples through the API, the database schema, and the frontend.

A NAMED CONSTANT makes the valid output set explicit and independent of the model file:

```
CLASSES = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]
# must match model.classes_, verified at startup
# note: "0" is intentionally absent in v1
```

This list belongs in the code, not only in the model. The frontend, the database schema, and the API documentation all reference it. If the model is swapped for one trained on different classes, the assertion described below fires loudly at startup rather than returning silently wrong labels for the rest of the service's lifetime.

THE FUNCTION SIGNATURES fall out of the contract:

```
def classify_batch(images: np.ndarray) -> list[dict]:
    """
    Args:
        images: (N, 28, 28) uint8, values 0-255
    Returns:
        List of N dicts:
        {"prediction": str, "confidence": float,
         "scores": dict[str, float]}
    """
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]
```

BATCH-FIRST DESIGN. What happens when 50 students submit a drawing in the same second? If the server processes each request individually, the model is called 50 times with a $(1, 784)$ array. If it batches them, the model is called once with a $(50, 784)$ array. sklearn's `predict_proba` is natively vectorised, so the cost is nearly identical. Design around batch; single-user is a special case.

```
def classify(image: np.ndarray) -> dict:
    """Single-image convenience wrapper."""
    return classify_batch(image[np.newaxis])[0]
```

The wrapper is for tests only. The API in Block 5 always calls `classify_batch`, even for a single drawing: it passes a $(1, 28, 28)$ array. When a request queue is added later to amortise inference cost, no signature changes are needed; the queue assembles a batch and the function consumes it.

Load `_pipeline` once at module level:

```
import joblib, os
_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"
```

Deserialising on every request kills throughput; the model stays in memory for the lifetime of the process. The assertion is the key teaching moment: it fires at startup if the model's classes do not match the code constant. A swapped model with the wrong classes is caught the moment the service starts, not the next time a user gets a wrong digit.

`MODEL_PATH` from `.env`. The path is configuration, not code. Swapping the model in production becomes an env-var change plus a service restart, not a code change and a deploy. This is the same pattern as the database URL in Block 6 and the API key from Block 3.

Exercise: The Classifier Module

IMPLEMENT THE INFERENCE INTERFACE IN `app/classifier.py` with the class constant, module-level loading, the startup assertion, and the two functions:

```
import os
import joblib
import numpy as np
from dotenv import load_dotenv

load_dotenv()

CLASSES = ["1","2","3","4","5","6","7","8","9"]

_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"

def classify_batch(images: np.ndarray) -> list[dict]:
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]

def classify(image: np.ndarray) -> dict:
    return classify_batch(image[np.newaxis])[0]
```

UPDATE `.env` AND `.env.example` so the loader knows where to find the artefact:

```
MODEL_PATH=models/digit_classifier_v1.pkl
```

WRITE `predict.py` AS A SMOKE TEST. Load five MNIST test samples, call `classify_batch`, print the predictions next to the ground truth:

THE ASSERTION IS THE HEADLINE. Comment it out, save, restart Python, and import the module. Now mutate `CLASSES` to add a fake `"10"` entry, restore the assertion, restart, and observe the assertion firing. The startup error message is exactly what you want to see when a model swap goes wrong.

```
from app.classifier import classify_batch
from sklearn.datasets import fetch_openml
import numpy as np

X, y = fetch_openml("mnist_784", version=1,
    return_X_y=True, as_frame=False)
images = X[:5].reshape(-1, 28, 28).astype(np.uint8)
truth  = y[:5]
results = classify_batch(images)
for r, t in zip(results, truth):
    print(f"Pred: {r['prediction']} "
          f"(conf {r['confidence']:.2f})  True: {t}")
```

Read `predict.py` as the API preview. This is what the API in Block 5 will do: take pixel data, call `classify_batch`, return the result. Block 5 swaps stdin for HTTP and stdout for JSON, and moves the service to `prod`; the inference logic itself is unchanged.

Run the smoke test on dev and commit:

```
python predict.py
git add app/classifier.py predict.py \
    models/MODELCARD.md .env.example
git commit -m "Add classifier module and model card"
git push
```

The repository is now at the state `v0.3` marks: a working classifier, a dev-side smoke test, a model card, and an updated configuration contract. The `.pkl` stays on each developer's machine, the artefact path is in `.env`, and the rest of the team can reproduce the environment from the code in Git. Block 5 takes this dev-verified module and exposes it as an HTTP service on `prod`.

Optional: Add the Missing 0

THE DIGIT CLASS "0" WAS INTENTIONALLY EXCLUDED FROM V1. With v1 integrated and dev-verified, you can publish your own v2. Fork the course model repository so you have a place to push:

```
gh repo fork schutera/pixelwise-model --clone --remote
cd pixelwise-model
```

The fork already contains `train.py`, `digit_classifier_v1.pkl`, and `MODELCARD.md` from v1.0. Add `"0"` to the class list, include MNIST's class-0 samples in the training split, re-run `python train.py`, and save the new artefact as `digit_classifier_v2.pkl`. Update `MODELCARD.md` to document v2 with the new class, commit artefact, script changes, and card together, and cut a fresh tag:

WHY FORK INSTEAD OF BRANCHING UPSTREAM? Forking is the standard pattern for publishing a derivative of someone else's repository. You get write access to your fork, the upstream stays canonical, and a pull request is the natural way to upstream improvements later.

```
git add digit_classifier_v2.pkl train.py MODELCARD.md
git commit -m "v2: adds class 0"
git tag -a v2.0 -m "Adds class 0"
git push --follow-tags
```

On the PixelWise side, bump `MODEL_REPO=<you>/pixelwise-model` and `MODEL_VERSION=v2.0` in `.env` and re-run `gh release download`. The assertion will break the moment v2 is loaded against v1's `CLASSES`; fix it by updating the constant.

This challenge connects directly to Block 8: v2 is deployed via the feature flag, and Block 9's analytics compare v1 and v2 performance. No data collection is needed; MNIST has the 0s already.

COULD WE HAVE SPOTTED THE MISSING CLASS AT RUNTIME?

- After a few hundred predictions, a missing class is invisible: the model never predicts it, users never see an error, the database just never accumulates any 0s.
- When the model sees an input it was not trained on, it is often less certain. A systematic drop in average confidence could indicate out-of-distribution inputs: a weak signal, but a runtime signal that requires no labelled data.
- Returning `null` when `confidence < threshold` turns low confidence into a monitoring signal: the volume of rejected predictions is itself a metric.

A NOTE ON WHAT COMES NEXT. `app/classifier.py` works on dev, but it is trapped in a Python script. A user with a browser cannot call it; a teammate on a different machine cannot reach it; `prod` has not yet seen this artefact. The next block wraps `classify_batch` in

SYSTEMATIC DETECTION OF SILENT FAILURES is the domain of MLOps: uncertainty estimation, distribution shift, missing classes, silent degradation. This surface is intentionally shallow here. A dedicated MLOps course covers it in depth.

an HTTP endpoint, defines the request and response contract with Pydantic, and runs the result as a managed `systemd` service on `prod` so the model becomes reachable without a human in the loop.

Self-Reflection and Recap

Self-Reflection questions to guide your thoughts during and after the exercises:

- What is train/serve skew, and why is it the most common production ML defect?
- Why do we load the model once at module level instead of on every request?
- What does the startup assertion protect against, and what would happen without it?
- Why is the `.pkl` in `.gitignore` but `MODELCARD.md` is committed?
- How would you detect at runtime that the model is missing a class it should predict?
- Why is batch-first design the right default, even when most requests carry a single image?

Recap of key concepts:

- A `.pkl` file is a build artefact, not source code; never commit it to Git.
- Train/serve skew is silent by default; the startup assertion is the loudest defence.
- Batch-first design scales from one user to N concurrent users with no code changes.
- Module-level loading keeps the model in memory for the lifetime of the process.
- The model card documents capabilities, limitations, and provenance, and updates with the model.
- `predict.py` proves end-to-end correctness before any API exists.

Milestone. `app/classifier.py` exposes a batch-first inference interface that scales from one student to N concurrent users with no code changes. `predict.py` proves it works end to end on real MNIST samples. `MODELCARD.md` documents what the model is and is not. The next chapter wraps the inference function in a FastAPI service so the model becomes reachable over HTTP.

Security thread. Never load a `.pkl` from an untrusted source; `joblib` and `pickle` deserialisation execute arbitrary code. `MODEL_PATH` comes from `.env`, never hard-coded, so swapping the model in production requires only an env-var change, not a code change. Training data and model artefacts are not committed to Git.

Teaser. The classifier is callable from Python, but who, outside your terminal, can reach it?